

MAREVLO · PRODUCTION RAG

Scaling & the Real Costs of Vector Search

Everything looks cheap and fast in a notebook. Here is what changes — and what it costs — when the numbers get large.

PART III — PRODUCTION

Module 10

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

In the last module we made our system as good as it could be at small scale, measuring every change carefully. But "good" and "good at scale" are different questions with different answers. A system that is fast, cheap, and accurate over a hundred documents on your laptop can become slow, expensive, and surprisingly inaccurate over a million documents serving a thousand users. This module is about that gap. It is less about adding new features and more about **reasoning** — understanding the costs that only appear at scale, the trade-offs that only become real when the numbers grow, and the practical skill of estimating what your system will cost *before* you build it.

This is the kind of knowledge that separates a system that survives its own success from one that collapses under it. Many RAG projects work beautifully in a demo and then fall apart in production — not because the idea was wrong, but because nobody asked what would happen at a thousand times the scale. By the end of this module you will know what to ask, and how to answer it with numbers rather than hope.

THE WHOLE MODULE IN ONE SENTENCE

At scale you pay in four currencies — the one-time cost of embedding, the ongoing cost of storage, the per-query cost of every search and answer, and the cost of latency — and the most dangerous mistakes are validating on a toy corpus and never modelling those costs before you commit.

Where we are

We are entering Part III, production. The system works and is well-tuned; now we make sure it can run for real, at real size, at a cost that makes sense. This module gives you the reasoning; the next one (Module 11) puts it into practice with a production-grade database. Nothing here changes how the system answers — it changes how clearly you can see what running it actually demands.

SECTION 1

The notebook lies to you

At small scale, every cost is invisible and every answer looks easy. That comfort is exactly what makes the notebook dangerous.

When you build and test on a small collection — a hundred documents, a few test questions, one user at a time — three comforting illusions take hold, and all three break at scale. The first illusion is that it is **cheap**: embedding a hundred chunks costs almost nothing, and a handful of test queries cost pennies, so cost never enters your mind. The second is that it is **fast**: with so little data, every search is instant, so latency seems like a non-issue. The third, and most subtle, is that it is **accurate**: with only a hundred chunks, the right chunk for any question has almost no competition, so retrieval looks flawless.

Every one of these illusions is a property of small scale, not of your system. Multiply the data by ten thousand and the users by a thousand, and cost becomes a real bill, latency becomes something users feel, and the right chunk now competes against thousands of near-misses. The cruel part is that none of this shows up in the demo, so teams ship

the demo, and discover the truth in production with customers watching. The cure is to stop trusting the notebook and start reasoning about scale deliberately — which is what the rest of this module teaches.

THINK ABOUT IT

Your demo answers a test question in half a second and costs a fraction of a cent. It feels done. But ask: if ten thousand users each asked five questions a day, what would the monthly bill be? Would the half-second hold when two hundred of them ask at once? Would the right chunk still win against a million competitors instead of ninety-nine? If you cannot answer these from your demo — and you cannot — then the demo has told you almost nothing about whether the system can actually run.

SECTION 2

The four currencies you pay in

Running RAG at scale costs you in four distinct ways. Knowing which is which — and which one explodes — is the heart of cost reasoning.

To reason about cost, you first have to see that there is not one cost but four, and they behave very differently as you grow. Two are about money you pay regardless of how busy the system is, one is about money that grows with every question asked, and one is not money at all but the time users wait. Let us take them in turn, and then see them together in a picture.

The first currency is the **embedding cost of indexing**. To build your store, every chunk must be turned into an embedding, and for a paid embedding model that costs money. The important thing about this cost is that you pay it *once* per indexing — unless you change your embedding model, in which case you must re-embed everything from scratch, which is why we warned so strongly in Module 3 about choosing carefully. For a million chunks this is a real but one-time expense.

The second currency is **storage**. Every embedding is a long list of numbers that must be stored, and there are a lot of them. A million chunks, each a 1,536-number embedding, is several gigabytes of vectors — and managed databases charge you monthly for that space. This cost is roughly flat: it depends on how much data you have stored, not on how busy you are, so it ticks along every month whether you serve ten queries or ten million.

The third currency is the **per-query cost**, and this is the one that matters most, because it grows with every single question your users ask. Each query embeds the question (cheap), searches the store (cheap per query), and — the expensive part — calls the language model to generate an answer. That model call, which reads your instructions and context and writes a reply, is almost always the dominant cost of a query. Multiply it by thousands of queries a day, every day, and this is the currency that can quietly bankrupt a product.

The fourth currency is **latency** — not money, but the time a user waits for an answer. It is dominated by the same model call, and it gets worse when many users arrive at once and their requests contend for the same resources. Latency does not appear on a bill, but it appears in whether users stay.

When each cost hits — and which one grows with traffic



Figure 1. The four costs by when they hit. Embedding is one-time, storage is flat monthly, but the per-query cost — dominated by the model call — grows with every question and is the one that can sink a product at scale.

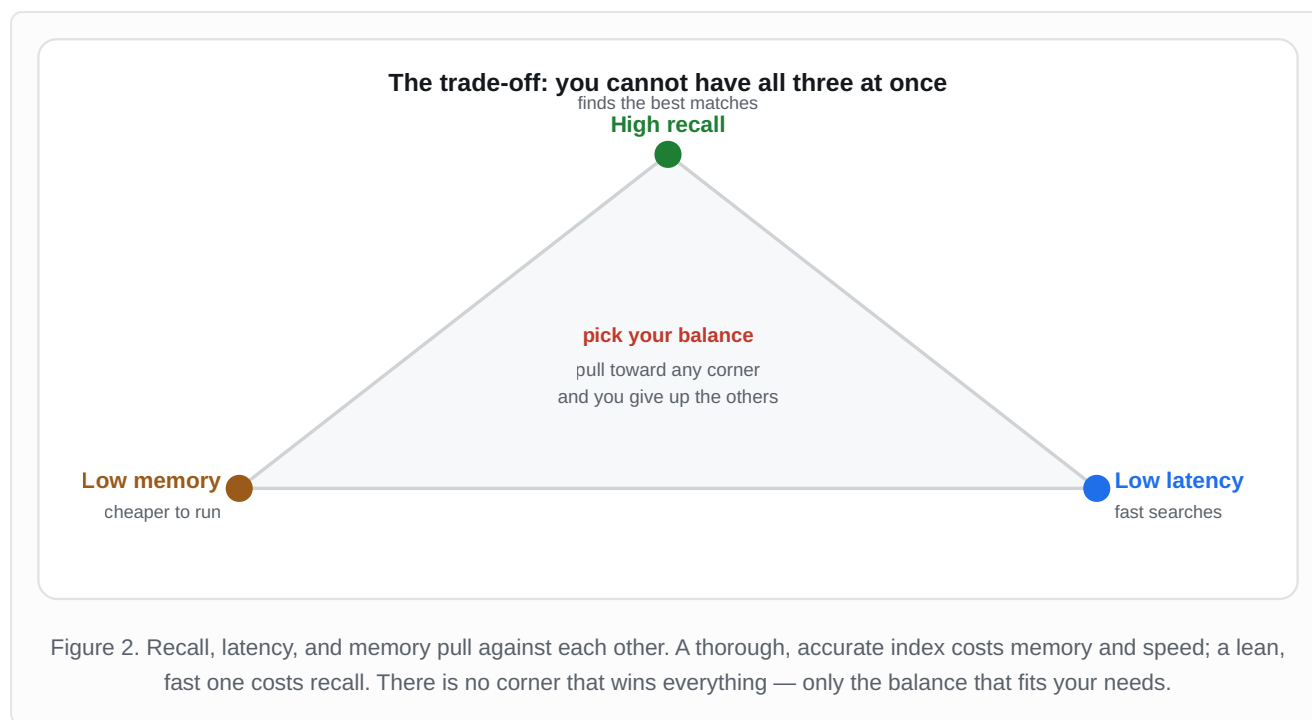
SECTION 3

Why search is "approximate" — and the trade-off you control

At scale, exact search becomes impossible, so vector databases approximate. That approximation comes with a dial you must understand.

Back in Module 4 we mentioned that vector databases use *approximate* nearest-neighbour search, and promised to explain the trade-off properly once scale was on the table. Now it is, so here is the full picture. Exact search — comparing your query against every single stored vector — is perfectly accurate but, at a million or a billion vectors, far too slow for every query. So databases build a clever index that lets them check only a promising subset, which is enormously faster but means they can occasionally miss the true nearest vector and return the second-best instead. That is what "approximate" means.

The crucial thing to understand is that this approximation is not fixed — it is a **dial you control**, and the dial connects three things that pull against each other: *recall* (how often the search finds the genuinely best matches), *latency* (how fast each search is), and *memory* (how much it costs to hold the index). You cannot maximise all three at once. Building a more thorough index and searching it more carefully improves recall but uses more memory and makes each search slower. Trimming the index to save memory and speed up search costs you some recall. There is no setting that is best at everything; there is only the balance that is right for your needs.



ANALOGY — SEARCHING A LIBRARY

Imagine finding the most relevant book in a huge library. You could read every single book to be certain — perfect recall, impossibly slow. Or you could rely on a good catalogue that points you to the right shelf and check only those few books — much faster, and almost always right, but just occasionally the perfect book was mis-shelved and you miss it. A more detailed catalogue (more memory) helps you miss fewer, but takes

more space and time to consult. The approximate search dial is exactly this: how much catalogue effort to spend versus how fast and cheap to stay.

You do not need to memorise the specific settings, because they differ between databases and have technical names. What matters is the principle: when someone is alarmed that a known chunk occasionally fails to appear, the usual cause is not a bug but the recall setting on this dial — and it can be turned up, at the price of some speed or memory. Knowing the dial exists turns a frightening mystery into a deliberate choice.

MEDIUM CONFIDENCE ON SPECIFICS — THEY ARE DATABASE-DEPENDENT

SECTION 4

The scale cliff: why quality quietly drops as you grow

The same retrieval that was flawless at a hundred chunks can stumble at a million — and the techniques from earlier modules are what rescue it.

There is a quieter danger at scale than cost or speed, and it catches the most teams: answer quality can silently *fall* as your collection grows, even though you changed nothing. Here is why. At a hundred chunks, the one chunk that answers a question has almost no competition — it stands out easily and retrieval looks perfect. At a million chunks, that same chunk now competes against thousands of others that are *nearly* as relevant, and small weaknesses in your chunking or embeddings that were harmless at small scale now let a near-miss edge out the true best chunk. Nothing broke; the crowd simply got large enough that being roughly right is no longer good enough.

This is exactly why the techniques from Modules 7 and 9 exist, and why we said some of them "only matter at scale." Hybrid search, metadata filtering, and re-ranking are all ways of cutting through a large crowd to find the genuinely best chunk. At a hundred chunks they add little, because the right answer already wins. At a million, they are often the difference between a system that works and one that returns plausible near-misses. So the practical lesson is twofold: validate your system at *realistic* scale, not on a toy corpus that flatters it, and expect to lean harder on hybrid search, filtering, and re-ranking the larger your collection grows.

SECTION 5

The skill that matters most: modelling cost before you build

A few minutes of arithmetic before you commit can save a product from launching with economics that do not work.

Here is the single most valuable practical skill in this module. Before you commit to an architecture, estimate what it will cost at the scale you actually expect — at a thousand queries a day, ten thousand, and a hundred thousand. This is simple arithmetic, and it routinely reveals that a design which felt fine in the demo would lose money on every user once real traffic arrived. It is far better to learn that on paper than from your bank balance.

The method is straightforward. Take the per-query cost — dominated, as we saw, by the model call — and multiply it by your expected daily queries to get a daily cost, then by thirty for a monthly figure. Add the flat monthly storage cost. The one-time indexing cost you note separately, since it does not recur. Below is a small estimator that does this; the prices in it are illustrative round numbers to show the method, and you must replace them with the real prices of the services you actually use.

LAB 10.1 — A cost estimator (our own — the numbers are illustrative)

```
# IMPORTANT: these prices are illustrative round numbers to show the METHOD.
# Always replace them with the real, current prices of YOUR services.

PRICE_PER_INPUT_TOKEN = 0.15 / 1_000_000 # e.g. $0.15 per million input tokens
PRICE_PER_OUTPUT_TOKEN = 0.60 / 1_000_000 # e.g. $0.60 per million output tokens
STORAGE_PER_GB_MONTH = 0.25 # e.g. $0.25 per GB per month

def cost_per_query(input_tokens=1800, output_tokens=200):
    """The model cost of answering one question.

    input_tokens = instructions + retrieved context + question (~1800)
    output_tokens = the answer the model writes (~200)
    This model call is almost always the dominant per-query cost.
    """
    return (input_tokens * PRICE_PER_INPUT_TOKEN +
            output_tokens * PRICE_PER_OUTPUT_TOKEN)

def monthly_cost(queries_per_day, num_chunks=1_000_000, dims=1536):
    """Estimate the monthly running cost at a given traffic level."""
    # Per-query cost grows directly with traffic -- the currency that explodes.
    query_cost = cost_per_query() * queries_per_day * 30

    # Storage is roughly flat: vectors are num_chunks * dims * 4 bytes.
    gigabytes = (num_chunks * dims * 4) / 1_000_000_000
    storage_cost = gigabytes * STORAGE_PER_GB_MONTH

    return {"queries": round(query_cost, 2),
            "storage": round(storage_cost, 2),
            "total": round(query_cost + storage_cost, 2)}

for daily in [1_000, 10_000, 100_000]:
    print(daily, "queries/day ->", monthly_cost(daily))
```

Run that and a clear, sobering pattern emerges. The storage cost stays roughly the same regardless of traffic, while the per-query cost grows in step with how busy you are — so as you scale up, the model call comes to dominate everything else. The chart below shows the shape, using the illustrative numbers above.

Illustrative monthly cost as traffic grows (note: each step is 10x)

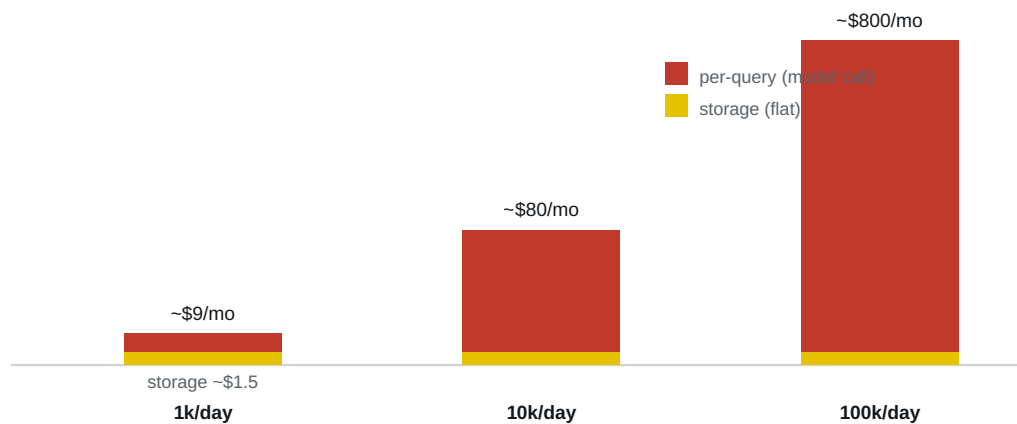


Figure 3. Storage stays tiny and flat; the per-query model cost grows directly with traffic and quickly dominates. At a free or low-priced B2C product, this is the number that decides whether the business works. (Illustrative figures — use your real prices.)

THE DECISION THIS SKILL PROTECTS

At a hundred thousand queries a day, an "almost free" half-cent per query becomes a serious monthly bill. If your product is free to users, that bill comes entirely out of your pocket, and the per-query cost effectively sets a ceiling on how many free users you can afford. Modelling this *before* you build tells you whether your business model and your architecture can coexist — a question far cheaper to answer on paper than after launch.

HIGH CONFIDENCE ON THE PRINCIPLE; FIGURES ARE ILLUSTRATIVE

SECTION 6

The levers that control cost and latency

Once you can see the costs, here are the dials that bring them down — most of them callbacks to techniques you already know.

The good news is that almost everything you have already learned doubles as a cost-control lever, because the per-query model call is both the biggest cost and the biggest source of latency. Reduce what that call has to do, and you reduce both at once.

Caching — usually the single biggest win

Store answers to repeated or very similar questions so you do not pay the model to answer the same thing twice. In most real systems a meaningful share of questions repeat, and serving those from a cache is both free and instant.

A smaller or cheaper model where quality allows

Test a cheaper model against your evaluation set. If it scores nearly as well, the cost saving on every query may be enormous. Let the eval set, not fear, decide.

Tighter token budgets (Module 7)

Send less context. Every token you trim from the prompt is paid back on every single query, forever. Re-ranking helps here too, by letting a few excellent chunks replace many mediocre ones.

Smaller embedding dimensions (Module 3)

Fewer dimensions mean less storage and faster search. If a smaller embedding holds its quality on your data, it lowers the flat storage cost and speeds every query.

The recall dial (Section 3)

Tune the approximate-search setting to the balance of accuracy, speed, and memory your system actually needs, rather than paying for more thoroughness than your answers require.

FAULTS AND CORRECTIONS

The scaling and cost mistakes that sink products

For each: the broken habit, why it bites at scale, and the fix.

FAULT 1 — Validating on a toy corpus, then shipping to full scale

THE BROKEN HABIT

```
# The eval set was built and tuned over 100 documents. It scores 95%.  
# We ship to the full 500,000-document collection assuming it will hold.  
deploy(system) # validated only at toy scale
```

WHY IT BITES AT SCALE

At a hundred documents the right chunk has no competition, so retrieval looks flawless — a property of the small size, not your system. At full scale the right chunk competes with thousands of near-misses, and the quality you measured evaporates. You validated a situation that will never exist in production, so your 95% told you almost nothing about the real thing.

THE CORRECTION

```
# Build and run the eval set against a realistically-sized collection.  
# Quality at toy scale does not predict quality at real scale.  
load_full_corpus(system)  
score = run_eval(eval_set) # measured where it actually matters
```

FAULT 2 — Never modelling cost before committing the architecture

THE BROKEN HABIT

```
# "It costs a fraction of a cent per query, so cost is not a concern."  
# Built and launched a free product on a premium model. Then the bill arrived.  
launch_free_product(using=premium_model)
```

WHY IT BITES AT SCALE

A fraction of a cent per query feels free until you multiply by a hundred thousand queries a day for thirty days. For a free-to-user product, that entire bill is yours, and it may exceed any revenue the product could ever generate. Discovering this after launch means re-architecting under pressure, or shutting down. The arithmetic that would have warned you takes five minutes.

THE CORRECTION

```
# Model the monthly cost at your expected scale BEFORE you commit.  
for daily in [1_000, 10_000, 100_000]:  
    print(daily, monthly_cost(daily)) # find the scale where it stops working
```

FAULT 3 — Cranking recall to the maximum and ignoring the cost

THE BROKEN HABIT

```
# "More accurate is always better" -- so set the index to its most  
# thorough recall setting, ignoring memory and latency entirely.  
index.build(thoroughness="maximum") # huge memory, slow searches
```

WHY IT BITES AT SCALE

Recall, latency, and memory pull against each other (the triangle in Section 3). Pushing recall to the maximum can multiply your memory cost and slow every search, often to chase a last sliver of accuracy your answers do not even need. You pay a steep, ongoing price for thoroughness that makes no visible difference to users.

THE CORRECTION

```
# Choose the recall setting that meets your quality bar on the eval set,  
# then stop. Measure recall AND latency AND memory together.  
# Tune up only if the eval score actually needs it.
```

FAULT 4 — No caching, so you pay full price for repeated questions

THE BROKEN HABIT

```
# Every question, even an identical one asked a thousand times, runs the  
# full pipeline and pays the full model cost again.  
answer = answer_question(question) # no cache, ever
```

WHY IT BITES AT SCALE

In real systems a meaningful share of questions repeat — the same few popular questions asked over and over. Without a cache, you pay the full model cost and the full latency for each repeat, as if you had never answered it before. At scale this is a large, completely avoidable expense, and it slows answers that could have been instant.

THE CORRECTION

```
# Cache answers by question, so repeats are free and instant.  
cache = {}  
def cached_answer(question):  
    if question not in cache:  
        cache[question] = answer_question(question) # pay once  
    return cache[question] # then free
```

FAULT 5 — Choosing the biggest model and biggest embeddings "to be safe"

THE BROKEN HABIT

```
# Defaulting to the largest, most expensive model and highest-dimension  
# embeddings without checking whether smaller ones would do just as well.  
model = most_expensive_model  
embedder = highest_dimension_embedder
```

WHY IT BITES AT SCALE

The biggest options feel safe, but "safe" here means paying the maximum on every query and every gigabyte forever, often for quality your task never needed. A smaller model or smaller embedding frequently scores within a hair of the largest on a real eval set, at a fraction of the cost. Bigger is not safer; it is just more expensive until proven necessary.

THE CORRECTION

```
# Start smaller and let the eval set justify any upgrade.  
score_small = run_eval(eval_set) # with the cheaper model  
# Only move to a bigger model if the score genuinely needs it AND the  
# extra cost is worth the gain.
```

FAULT 6 — Treating a model switch as free, forgetting the re-index cost

THE BROKEN HABIT

```
# A new embedding model scores 3% better on a leaderboard, so we switch --  
# forgetting that this invalidates every existing embedding.  
embedder = new_better_embedder # but the store holds the OLD embeddings
```

WHY IT BITES AT SCALE

As we learned in Module 3, changing the embedding model means every existing embedding is now on the wrong map, so the entire collection must be re-embedded from scratch. At a million chunks that is a real cost in money and time, plus a careful migration. A "free" 3% upgrade can turn into an expensive, risky project that a moment's thought would have priced in advance.

THE CORRECTION

```
# Before switching, price the full re-index and weigh it against the gain.  
chunks = 1_000_000  
reindex_cost = chunks * cost_to_embed_one_chunk() # the true cost of switching  
# Switch only if the measured quality gain justifies this one-time cost.
```

Checkpoint: try it yourself

This exercise builds the most important habit in the module — turning a vague worry about cost into a concrete number you can decide on. It takes about thirty minutes and needs no large dataset, only arithmetic and honest assumptions.

1. Use the estimator from Lab 10.1, but replace the illustrative prices with the real, current prices of the model and storage you would actually use. Getting the real numbers in is the whole point.
2. Estimate your monthly cost at 1,000, 10,000, and 100,000 queries per day. Write down all three. Notice how the storage barely moves while the per-query cost grows tenfold at each step.
3. Now decide your break-even point: if your product earns nothing per user, at what daily query volume does the bill become uncomfortable for you? That number is a real constraint on your business, and you just discovered it on paper instead of in production.
4. Finally, pick one lever from Section 6 — say, a cheaper model or a tighter token budget — estimate how much it would cut the per-query cost, and re-run the numbers. See how a single lever shifts the whole picture.

WHAT YOU SHOULD NOTICE

Cost stopped being a vague anxiety and became a number you can act on. You saw which currency explodes (per-query) and which barely matters (storage), and you watched a single lever move the whole bill. That shift — from worrying about cost to modelling it — is exactly the production mindset this module exists to build.

What to carry into Module 11

THE SIX THINGS TO REMEMBER

One, the notebook hides every cost that matters — small scale makes everything look cheap, fast, and accurate, and all three illusions break when you grow. **Two**, you pay in four currencies: one-time embedding, flat monthly storage, per-query cost, and latency — and the per-query model call is the one that explodes with traffic. **Three**, search is approximate because exact search cannot scale, and the approximation is a dial trading recall against latency and memory — you cannot maximise all three. **Four**, quality can silently fall as the collection grows, which is exactly why hybrid search, filtering, and re-ranking matter most at scale — so validate at realistic scale, never on a toy corpus. **Five**, model your cost at your expected traffic before you commit, because the per-query cost can quietly decide whether your business works. **Six**, the levers that cut cost — caching, cheaper models, tighter budgets, smaller dimensions, the recall dial — are mostly techniques you already know, because reducing the model's work cuts cost and latency together.

We now understand what running RAG at scale truly costs and how to keep those costs in check. The last piece is the home for all of this in production: a database built not for a notebook but for real durability, concurrency, and backups. In Module 11 we move from our learning store to a production-grade one — Postgres with pgvector — and put everything we have learned onto a foundation that can carry real traffic without falling over.